JSON – JavaScript Object Notation

Some of the slides taken from Douglas Crockford / Paypal Inc.

2008-2020 Copyright E. Horowitz, M. Papa

What is JSON

- **JSON**, short for **JavaScript Object Notation**, is a lightweight data interchange format.
  - It is a text-based, human-readable format for representing simple data structures and associative arrays (called objects).
- The JSON format is specified in RFC 4627 (July **2006**) authored by Douglas Crockford.
  - https://tools.ietf.org/rfc/rfc4627.txt
  - The official MIME type for JSON is application/json. The JSON file extension is .json.
- The JSON format is often used for transmitting structured data over a network connection in a process called serialization.
  - Its main application is in Ajax web application programming, where it serves as an alternative to the use of the XML format.
- Code for parsing and generating JSON data is readily available for a large variety of programming languages. The www.json.org website provides a comprehensive listing of existing JSON bindings, organized by language.

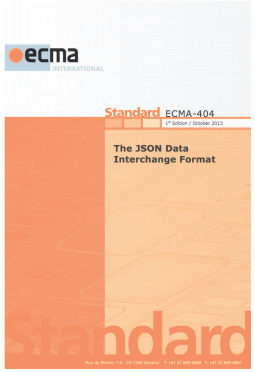
2008-2020 Copyright E. Horowitz, M. Papa

Brief History

- JSON was based on a subset of the JavaScript programming language (specifically, Standard ECMA-262 - now in its 11th Edition)
  - however it is considered to be a language-independent data format.
  - For the complete specification see

  https://www.ecma-international.org/publications/standards/Ecma-262.htm

  The JavaScript ECMA standard is based upon Netscape's JavaScript and Microsoft's Jscript



- Douglas Crockford was the original developer of JSON while he was at State Software, Inc. He is now Senior JavaScript Architect at Paypal
- http://www.json.org/, is a website devoted to JSON discussions and includes many JSON parsers

How to use the JSON format

- A JSON file allows one to load data from the server or to send data to it.
- Working with JSON involves three steps: (i) the browser processing, (ii) the server processing, and (iii) the data exchange between them.

1. **Client side (browser)**
   - The content of a JSON file (or stream), or the definition of JSON data is assigned to a variable, and this variable becomes an object of the program.
2. **Server side**
   - a JSON file (or stream) on the server can be operated upon by various programming languages, including PHP and Java thanks to parsers that process the file and may even convert it into classes and attributes of the language.
3. **Data exchange**
   - Loading a JSON file from the server may be accomplished in JavaScript in several ways:
   - directly including the file into the HTML page, as a JavaScript **.json external file**.
   - loading by a JavaScript command
   - using **XMLHttpRequest()**
   - To convert JSON into an object, it can be passed to the JavaScript **eval()** function.
   - Sending the file to the server may be accomplished by XMLHttpRequest(). The file is sent as a text file and processed by the parser of the programming language that uses it.

2008-2020 Copyright E. Horowitz, M. Papa

JSON and XMLHttpRequest Example

- **The XMLHttpRequest code:**

```
var req = new XMLHttpRequest();
req.open("GET", "file.json", true);
req.onreadystatechange = myCode; // the callback
req.send(null);
```

- **The JavaScript callback: eval() parses JSON, creates an object and assigns it to variable doc**

```
function myCode() {
 if (req.readyState == 4) {
   if (req.Status == 200) {
     var doc = eval('(' + req.responseText + ')');
 }
}
```

- **Using the data:**

```
var menuName = doc.getElementById('menu'); // finding a field menu
doc.menu.value = "my name is"; // assigning a value to the field
```

- **How to access data:**

```
doc.commands[0].title // read value of the "title" field in the array
doc.commands[0].action // read value of the "action" field in the array
```

JavaScript eval()

- The JavaScript **`eval()`** function evaluates a string and executes it as if it was JavaScript code, e.g.

```
<script type="text/javascript">
eval("x=10;y=20;document.write(x*y)");
document.write("<br />");
document.write(eval("2+2"));
document.write("<br />");
var x=10;
document.write(eval(x+17));
document.write("<br />"); </script>
```

- produces the output

```
200
4
27
```

- Because JSON-formatted text is also syntactically legal JavaScript code, an easy way for a JavaScript program to parse JSON-formatted data is to use the built-in JavaScript eval() function
- the JavaScript interpreter itself is used to *execute* the JSON data to produce native JavaScript objects.
- The eval() technique is subject to security vulnerabilities if the data and the entire JavaScript environment is not within the control of a single trusted source; See:
- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/eval

JSON Basic Data Types

- ***String*** (double-quoted unicode with backslash escaping)
- ***Numbers*** (integer, real, or floating point)
- ***Booleans*** (true and false)
- ***Object*** (collection of key:value pairs, comma-separated and enclosed in curly brackets)
- ***Array*** (an ordered sequence of values, comma-separated and enclosed in square brackets)
- ***Null*** (a value that isn't anything)

2008-2020 Copyright E. Horowitz, M. Papa

String

- Sequence of 0 or more Unicode characters
- No separate character type
  - A character is represented as a string with a length of 1
- Wrapped in "double quotes"
- Backslash escapement

2008-2020 Copyright E. Horowitz, M. Papa

Object

- Objects are unordered containers of key/value pairs
- Objects are wrapped in **{ }**
- **,** separates key/value pairs
- **:** separates keys and values
- Keys are strings
- Values are JSON values
- Can be used to represent: struct, record, hashtable, object

2008-2020 Copyright E. Horowitz, M. Papa

Example Object


```
{"name":"Jack B. Nimble","at large":
true,"grade":"A","level":3,
"format":{"type":"rect","width":1920,
"height":1080,"interlace":false,
"framerate":24}}
```

Example Object Formatted

```
{
    "name":     "Jack B. Nimble",
    "at large": true,
    "grade":    "A",
    "level": 3,
    "format": {
        "type":      "rect",
        "width":     1920,
        "height":    1080,
        "interlace": false,
        "framerate": 24
    }
}
```

2008-2020 Copyright E. Horowitz, M. Papa

Array

- Arrays are <u>ordered</u> sequences of values
- Arrays are wrapped in **[ ]** (square brackets)
- **,** separates values
- JSON does not talk about indexing.
  - An implementation can start array indexing at 0 or 1.

Two Examples of JSON Arrays

- **One dimensional**

```
["Sunday", "Monday", "Tuesday", "Wednesday",
  "Thursday", "Friday", "Saturday"]
```

- **Two dimensional**

```
[
   [0, -1, 0],
   [1, 0, 0],
   [0, 0, 1]
]
```

Arrays vs Objects

- Use objects when the key names are arbitrary strings
- Use arrays when the key names are sequential integers

2008-2020 Copyright E. Horowitz, M. Papa

JSON is Not XML

JSON

- Objects
- Arrays
- Strings
- Numbers
- Booleans
- null

XML

- element
- attribute
- Attribute string
- content
- <![CDATA[ ]]>
- Entities
- Declarations
- Schema
- Stylesheets
- Comments
- Version
- namespace

2008-2020 Copyright E. Horowitz, M. Papa

JSON vs. XML Example

JSON

```
{
  "menu": "File",
  "commands": [
    {
      "title": "New",
      "action":"CreateDoc"
    },
    {
      "title": "Open",
      "action": "OpenDoc"
    },
    {
      "title": "Close",
      "action": "CloseDoc"
    }
  ]
}
```

XML Equivalent

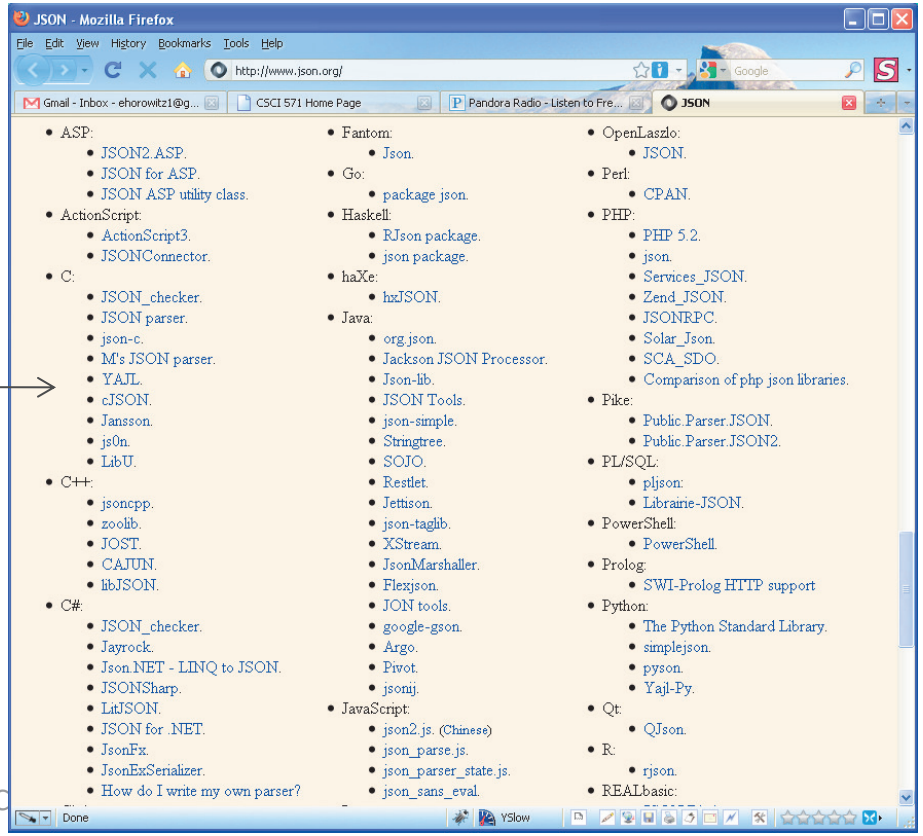
```
<?xml version="1.0" ?>
 <root>
   <menu>File</menu>
   <commands>
     <item>
       <title>New</value>
       <action>CreateDoc</action>
     </item>
     <item>
       <title>Open</value>
       <action>OpenDoc</action>
     </item>
     <item>
       <title>Close</value>
       <action>CloseDoc</action>
     </item>
   </commands>
 </root>
```

Rules for JSON Parsers

- A JSON decoder must accept all *well-formed* JSON text
- A JSON decoder may also accept non-JSON text
- A JSON encoder must only produce well-formed JSON text
- A list of decoders for JSON can be found at

http://www.json.org/

JSON parsers for programming languages include: C, C++, C#, Java, JavaScript, Perl, PHP



Same Origin Policy

- Same origin policy is a security feature that browsers apply to client-side scripts
- It prevents a document or script loaded from one "origin" from getting or setting properties of a document from a different "origin"
  - Rationale: the browser should not trust content loaded from arbitrary websites
- Given the URL: http://www.example.com/dir/page.html

URL	Outcome	Reason
http://www.example.com/dir2/other.json	Success	Same protocol and host
http://www.example.com/dir/inner/other.json	Success	Same protocol and host
http://www.example.com:81/dir2/other.json	Failure	Same protocol and host but different port
https://www.example.com/dir2/other.json	Failure	Different protocol
http://en.example.com/dir2/other.json	Failure	Different host
http://example.com/dir2/other.json	Failure	Different host

2008-2020 Copyright E. Horowitz, M. Papa

JSON: The Cross-Domain Hack

- JSON and the **<script> tag** provide a way to get around the Same Origin Policy

<script src=http://otherdomain.com/data.js>

</script>

- The **src** attribute of a script tag can be set to a **URL from any server**, and every browser will go and retrieve it, and read it into your page
- So a script tag can be set to point at a URL on another server with JSON data in it, and that JSON will become a global variable in the webpage
- So JSON can be used to grab data from other servers, without the use of a server-side proxy
- Available in **HTML** since 1994

MarkitOnDemand Example

- MarkitOnDemand provides Stock Lookup and Stock Quote APIs. Results are in JSON format.
- **Note: MarkitOnDemand assets have been sold the the API "retired"**
- To access the service you do not need an Application ID.
- Stock Lookup JSON REST call looks like this:

http://dev.markitondemand.com/MODApis/Api/v2/Lookup/json?input=AAP

- Response:


```
[{"Symbol":"AAP","Name":"Advance Auto Parts
Inc","Exchange":"NYSE"},{"Symbol":"AAP","Name":"Advance Auto Parts
Inc","Exchange":"Cboe BZX"},{"Symbol":"AAPL","Name":"Apple
Inc","Exchange":"NASDAQ"}]
```


- Stock Quote JSON REST call looks like this:

http://dev.markitondemand.com/MODApis/Api/v2/Quote/json?symbol=AAPL

- Response:


```
{"Status":"SUCCESS","Name":"Apple
Inc","Symbol":"AAPL","LastPrice":223.59,"Change":6.89000000000001,"ChangePe
rcent":3.17951084448547,"Timestamp":"Wed Sep 11 00:00:00 UTC-04:00
2019","MSDate":43719,"MarketCap":1010443456200,"Volume":43591712,"ChangeYTD
":157.74,"ChangePercentYTD":41.7459109927729,"High":223.71,"Low":217.73,"Op
en":218.07}
```


- MarkitOnDemand will return data in multiple formats including JSON and XML

Lookup Sample JSON Result

- Here is an example of the MarkitOnDemand Lookup API returning JSON
- http://dev.markitondemand.com/MODApis/Api/v2/Lookup/json?input=AAP


```
[{
    "Symbol": "AAP",
    "Name": "Advance Auto Parts Inc",
    "Exchange": "NYSE"
}, {
    "Symbol": "AAP",
    "Name": "Advance Auto Parts Inc",
    "Exchange": "BATS Trading Inc"
}, {
    "Symbol": "AAPL",
    "Name": "Apple Inc",
    "Exchange": "NASDAQ"
}, {
    "Symbol": "AVSPY",
    "Name": "AAPL ALPHA INDEX",
    "Exchange": "NASDAQ"
}, {
    "Symbol": "AIX",
    "Name": "NAS OMX Alpha   AAPL vs. SPY  Settle",
    "Exchange": "NASDAQ"
}]
```

Quote Sample JSON Result

- Here is an example of the MarkitOnDemand Quote API returning JSON
- http://dev.markitondemand.com/MODApis/Api/v2/Quote/json?symbol=AAPL


```
{
   "Status": "SUCCESS",
   "Name": "Apple Inc",
   "Symbol": "AAPL",
   "LastPrice": 106.01,
   "Change": 1.43000000000001,
   "ChangePercent": 1.36737425894053,
   "Timestamp": "Wed Mar 16 15:59:00 UTC-04:00 2016",
   "MSDate": 42445.6659722222,
   "MarketCap": 587781243830,
   "Volume": 2953715,
   "ChangeYTD": 105.26,
   "ChangePercentYTD": 0.712521375641269,
   "High": 106.31,
   "Low": 104.6,
   "Open": 104.62
}
```

XMLHttpRequest Compared to the Dynamic Script Tag

	XmlHttpRequest	Dynamic script Tag
Cross-browser compatible?	No	Yes
Cross-domain browser security enforced?	Yes (*)	No
Can receive HTTP status codes?	Yes	No (fails on any HTTP status other than 200)
Supports HTTP GET and POST?	Yes	No (GET only)
Can send/receive HTTP headers?	Yes	No
Can receive XML?	Yes	Yes (but only embedded in a JavaScript statement)
Can receive JSON?	Yes	Yes (but only embedded in a JavaScript statement)
Offers synchronous and asynchronous calls?	Yes	No (asynchronous only)

Dynamic script tag is used by internet advertisers who use it to pull their ads into a web page

The script tag's main advantages are that it is not bound by the web browser's cross-domain security restrictions and that it runs identically on more web browsers than XMLHttpRequest.

If your web service happens to offer JSON output and a callback function, you can easily access web services from within your JavaScript applications without having to parse the returned data

* CORS-compatible browsers allow cross-domain XMLHttpRequest

Arguments against JSON

- JSON doesn't have namespaces
- JSON has no validator
  - Every application is responsible for validating its inputs
- JSON is not extensible
  - But it does not need to be
- JSON is not XML
  - But a JavaScript compiler is a JSON decoder

Features that make JSON well-suited for data transfer

- It is both a human and machine-readable format;
- It has support for unicode, allowing almost any information in any human language to be communicated
- The format is self-documenting in that it describes structure and field names as well as specific values
- The strict syntax and parsing requirements allow the parsing algorithms to remain simple, efficient, and consistent
- JSON has the ability to represent the most general of computer science data structures: records, lists and trees

2008-2020 Copyright E. Horowitz, M. Papa

Eval() and JSON.parse() Security

- The eval() function is very fast. However, it can compile and execute any JavaScript program, so there can be security issues
- In general
  - your browser should not trust machines not under your absolute control
  - Your server must validate everything the client tells it
- To help guard the browser from insecure JSON input, use **JSON.parse()** instead of eval ; e.g. JSON.parse() is used this way

  var myObject = **JSON.parse**(JSONtext [, reviver]);
- The optional reviver parameter is a function that will be called for every key and value at every level of the final result. Each value will be replaced by the result of the reviver function. This can be used to reform generic objects into instances of pseudoclasses, or to transform date strings into Date objects.

More on JSON.parse()

- JSON.parse() is included in ECMAScript since 5th Ed. and all recent desktop browsers (Chrome, Firefox 3.5+, IE 8+, Opera 10.5+, Safari 4+) and all mobile browsers. JSON.parse() compiles faster than eval(). See:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON/parse

Browser compatibility

Update compatibility data on GitHub

	Chrome	Edge	Firefox	Internet Explorer	Opera	Safari	Android webview	Chrome for Android	Firefox for Android	Opera for Android	Safari on iOS	Samsung Internet	Node.js
parse	3	12	3.5	8	10.5	4	≤37	18	4	11	Yes	1.0	Yes

What are we missing?

Full support

More on JSON.parse (cont'd)

- In JavaScript there is a function called JSON.parse. It uses a single call to eval to do the conversion, guarded by a single regexp test to assure that the input is safe.
- The input object is traversed recursively, and various functions are called for each member of the object in post-order (i.e. every object is reviewed after all its members have been reviewed).
- For each member, the following occurs:
  - If reviewer returns a valid value, the member value is replaced with the value returned by reviewer.
  - If reviewer returns what it received, the structure is not modified.
  - If reviewer returns null or undefined, the object member is deleted.
- The reviewer argument is often used to transform JSON representation of ISO date strings into UTC format Date objects.
- Here is the original source code for JSON.parse

```
JSON.parse = function (text) {
return
(/^(\s|[,:{}\[\]]|"(\\["\\bfnrtu]|[^\x00-\x1f"\\])*"|-
?\d+(\.\d*)?([eE][+-]?\d+)?|true|false|null)+$/.test(text))
&& eval('(' + text + ')'); };
```

- According to Doug Crockford, "It is ugly, but it is really efficient. "
- For the actual source implementation and explanation of the behavior of the "reference implementation" of JSON.parse see
  - https://github.com/douglascrockford/JSON-js
  - (for applications that need to run on obsolete browsers)
- JQuery provides implicit ParseJSON() method and responseJSON property. See
  - http://api.jquery.com/jquery.parsejson/ and http://api.jquery.com/jQuery.ajax/

Douglas Crockford Discusses AJAX

- Go to
- https://www.youtube.com/watch?v=-C-JoyNuQJs
- Start video at -27 minutes and run it as long as it is interesting
- He gets to JSON at -13 minutes

Python Includes JSON functionality

- Python includes an encoder and a decoder. JSON functions include
  - json.dump – serialize object as JSON formatted string
  - *class* json.JSONDecoder - simple JSON decoder
  - *class* json.JSONEncoder – extensible JSON decoder for Python data structures
- See: https://docs.python.org/3/library/json.html

JSON	Python
object	dict
array	list
string	str
number (int)	int
number (real)	float
true	True
false	False
null	None

2008-2020 Copyright E. Horowitz, M. Papa

Compact Encoding JSON in Python

```
>>> import json
>>> json.dumps([1, 2, 3, {'4': 5, '6': 7}], separators=(',', ':’))
'[1,2,3,{"4":5,"6":7}]'
```

Decoding JSON in Python

```
>>> import json
>>> json.loads('["foo", {"bar":["baz", null, 1.0, 2]}]’)
['foo', {'bar': ['baz', None, 1.0, 2]}]
>>> json.loads('"\\"foo\\bar”’)
'"foo\x08ar’
>>> from io import StringIO
>>> io = StringIO('["streaming API"]’)
>>> json.load(io) ['streaming API’]
['streaming API']
```